

FireQ: Fast INT4-FP8 Kernel and RoPE-aware Quantization for LLM Inference Acceleration

Daehyeon Baek*, Jieun Choi*, Jimyong Son*
 Kyungmin Bin, Seungbeom Choi, Kihyo Moon, Minsung Jang, Hyojung Lee†
 Cloud Research Team, Samsung SDS

Abstract

As large language models become increasingly prevalent, memory bandwidth constraints significantly limit inference throughput, motivating post-training quantization (PTQ). In this paper, we propose FireQ, a co-designed PTQ framework and an INT4-FP8 matrix multiplication kernel that accelerates LLM inference across all linear layers. Specifically, FireQ quantizes linear layer weights and key-values to INT4, and activations and queries to FP8, significantly enhancing throughput. Additionally, we introduce a three-stage pipelining for the prefill phase, which modifies the FlashAttention-3 kernel, effectively reducing time-to-first-token in the prefill phase. To minimize accuracy loss from quantization, we develop novel outlier smoothing techniques tailored separately for linear and attention layers. In linear layers, we explicitly use per-tensor scaling to prevent underflow caused by the FP8 quantization scaling factor of INT4 quantization, and channel-wise scaling to compensate for INT4’s coarse granularity. In attention layers, we address quantization challenges posed by rotary positional embeddings (RoPE) by combining pre-RoPE and post-RoPE scaling strategies. FireQ significantly outperforms state-of-the-art methods, achieving 1.68x faster inference in feed-forward network layers on Llama2-7B and 1.26x faster prefill phase performance on Llama3-8B compared to QServe, with negligible accuracy loss.

1 Introduction

As model sizes and input sequence lengths increase, inference throughput is increasingly bottlenecked by memory bandwidth. (Lee et al., 2024; Gholami et al., 2024) Post-training quantization (PTQ) addresses this bottleneck by reducing memory footprint, thereby supporting larger batch sizes and longer sequences. However, quantization can introduce accuracy loss due to numerical instabilities, particularly from outlier values causing underflow or overflow in low-bit quantization. While existing outlier mitigation methods, such as rotation and scaling, help preserve accuracy, they often incur significant inference-time overhead.

To overcome these limitations, we propose FireQ, a fast INT4-FP8 general matrix multiplication (GEMM) kernel explicitly optimized for accelerating quantized model inference. FireQ integrates effectively with quantization strategies for linear and attention layers: linear layer quantization addresses underflow issues caused by FP8 scaling factors in INT4 quantization, while attention layer quantization employs the rotary positional embeddings (RoPE) (Su et al., 2021)-aware scaling to enhance accuracy.

To accelerate linear layer computations, FireQ employs a specialized INT4-FP8 GEMM kernel optimized for our mixed-precision quantization strategy. Specifically, weights and activations are quantized to INT4 and FP8 (W4A8-FP), respectively, with FP8 scaling factors. Our kernel leverages CUDA cores to perform in-register INT4-to-FP8 dequantization, significantly improving throughput.

*The first three authors contribute equally and are listed alphabetically. †Corresponding author.

To address underflow risks caused by the FP8 scaling factor, we implement per-tensor scaling. Additionally, channel-wise absmean scaling reduces inter-channel variance, compensating for INT4 quantization’s coarse granularity, maintaining accuracy without added computational cost.

For attention layers, FireQ quantizes keys and values to INT4 and queries to FP8 (KV4Q8-FP). INT4 quantization of the key matrix presents unique challenges due to pronounced outliers compared to queries and values (Lin et al., 2024b). Additionally, RoPE introduces nonlinear transformations, complicating standard quantization techniques. To address this, we adopt post-RoPE quantization, reducing computational overhead relative to pre-RoPE methods, which require additional quantization and dequantization steps around RoPE. We further introduce a two-stage RoPE-aware outlier smoothing strategy: pre-RoPE normalization handles stable channel pairs, and targeted post-RoPE scaling addresses outlier channels. This method effectively preserves quantization accuracy without compromising throughput.

To accelerate the prefill phase of attention mechanisms, FireQ modifies FlashAttention-3 (Shah et al., 2024) by employing a producer-consumer warpgroup structure with three-stage pipelining. The producer warpgroup asynchronously loads query, key, and value matrices into shared memory, transposing the value matrix for optimal computation. The consumer warpgroup operates in three overlapping stages: (i) query-key multiplication with FP16 accumulation, (ii) softmax computation in FP16 and FP8 quantization, and (iii) value aggregation using FP8-quantized softmax outputs, with FP32 accumulation reduced to BF16. This pipeline effectively enhances hardware utilization and significantly reduces time-to-first-token (TTFT) for the prefill phase in inference.

We evaluate FireQ on H100 GPUs and compare its performance with state-of-the-art frameworks, including QServe (Lin et al., 2024b), Atom (Zhao et al., 2024), QuaRot (Ashkboos et al., 2024), and TensorRT-LLM (W4A8-FP, AWQ) (NVIDIA, 2023; Lin et al., 2024a). Our results demonstrate significant throughput improvements: FireQ achieves 1.8× higher throughput than QServe and 1.24× higher throughput than TensorRT-LLM for feed-forward network layers on the Llama2-7B model (batch size 16). For the Llama3-8B model with batch size 16 and sequence length 1024, FireQ also delivers a 1.26× speedup over QServe in the prefill phase. Despite these substantial throughput gains, FireQ maintains zero-shot accuracy comparable to existing frameworks, demonstrating an effective balance between inference efficiency and accuracy. Our ablation study further validates that the proposed quantization outlier smoothing strategies significantly contribute to reducing accuracy loss.

2 Background

2.1 Post-training Quantization

Post-training quantization (PTQ) encodes model weights and activations into lower-precision formats to reduce memory usage and computational overhead, without requiring model retraining. Specifically, symmetric uniform b -bit integer quantization converts a real-valued tensor $\mathbf{x} \in \mathbb{R}^n$ to integer values $\hat{\mathbf{x}} \in \{-2^{b-1}, \dots, 2^{b-1} - 1\}^n$ using a scaling factor σ :

$$\sigma = \frac{\max_i |x_i|}{2^{b-1} - 1}, \quad \hat{x}_i = \lfloor \frac{x_i}{\sigma} \rfloor, \quad x_i^{(q)} \approx \sigma \cdot \hat{x}_i, \quad (1)$$

where $x_i^{(q)}$ denotes the dequantized approximation. In FireQ, weights in linear layers and key-value (KV) matrices in attention layers are quantized to INT4 with FP8 scaling factors. Activations, on the other hand, are quantized directly into FP8 format (e.g., E4M3) using a BF16 scaling factor β :

$$\hat{y}_i = \text{FP8}(y_i/\beta), \quad y_i^{(q)} \approx \beta \cdot \hat{y}_i. \quad (2)$$

Quantization granularity differs by component: weights in linear layers use per-channel group-wise symmetric quantization, while keys and values in attention layers apply per-token quantization, and queries utilize FP8 precision.

2.2 Challenges in LLM Quantization

Quantizing LLMs introduces unique challenges due to their scaling, sensitivity to quantization noise, and complex transformer-based architecture. LLM inference operates autoregressively, generating tokens through iterative transformer blocks consisting of linear layers and multi-head attention layers:

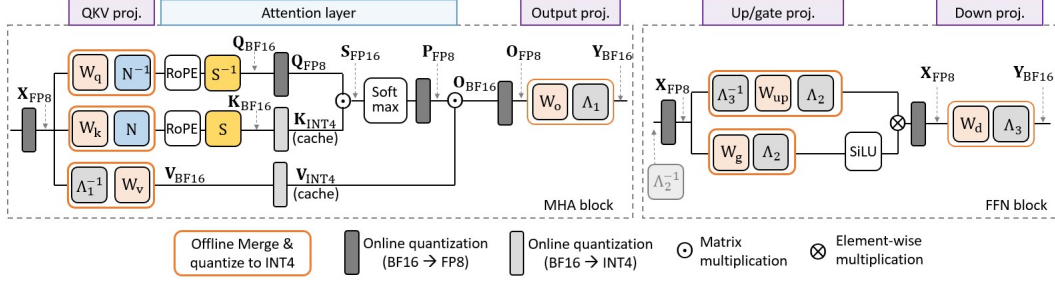


Figure 1: Overall diagram of mixed-precision quantization and corresponding outlier smoothing strategies. Λ , N , and S denote channel-wise absmean scaling (CAS), RoPE-preserving normalization (RPN), and Channel-wise RoPE scaling (CRS), respectively. All weights are merged with calibration matrices and quantized offline as shown in orange boxes, incurring no runtime overhead.

Linear layers. Linear layers perform general matrix multiplication between input activations and pre-trained weights, which covers up, down, gate projections in the feed-forward network (FFN) block, and query-key-value (QKV) generation, output projection in the multi-head attention (MHA) block, as shown in Figure 1. A typical linear operation is defined as:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^T, \quad (3)$$

where $\mathbf{X}$ and $\mathbf{Y}$ are the input and output matrices, respectively, and $\mathbf{W} \in \mathbb{R}^{N_{\text{out}} \times N_{\text{in}}}$ represents weights for input N_{in} and output N_{out} dimension.

Attention layer. Attention layer calculates attention scores between query (Q) and key (K) vectors and computes a weighted sum of value (V) vectors. Given query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ for a single head, the attention output $\mathbf{O}$ is computed as:

$$\mathbf{S} = \frac{1}{\sqrt{d}} \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}. \quad (4)$$

Outputs from multiple heads are concatenated and projected through an output layer.

Rotary positional embeddings (RoPE). RoPE injects position information into query and key vectors after their linear projection, enabling position-aware attention. Given a query or key matrix $\mathbf{x}^t \in \mathbb{R}^{N \times d}$ at position t , RoPE applies a 2D rotation $R(\theta_i^t)$ to pairs of channel $(\mathbf{x}_i^t, \mathbf{x}_j^t)$, where $j = i + d/2$ and $i = 0, \dots, d/2 - 1$. The transformed vector $(\tilde{\mathbf{x}}_i^t, \tilde{\mathbf{x}}_j^t)$ are computed by:

$$(\tilde{\mathbf{x}}_i^t, \tilde{\mathbf{x}}_j^t) = (\mathbf{x}_i^t, \mathbf{x}_j^t) \cdot R(\theta_i^t), \quad \text{and} \quad R(\theta_i^t) = \begin{pmatrix} \cos \theta_i^t & -\sin \theta_i^t \\ \sin \theta_i^t & \cos \theta_i^t \end{pmatrix}. \quad (5)$$

While RoPE effectively preserves the relative positional information by maintaining dot-product structure between queries and keys, it introduces complexity in quantization due to the mixing of paired channel distributions. This can cause instability, especially in low-bit quantization scenarios with pronounced outliers.

2.3 Hardware Background: Hopper GPU and FlashAttention-3

NVIDIA’s Hopper GPU (H100) features FP8-compatible tensor cores optimized for high-throughput, low-precision matrix multiplication, facilitating efficient inference of quantized LLMs. These cores natively support FP8 formats and achieve double throughput for FP16/BF16 operations, which is particularly beneficial during epilogue computations. FlashAttention-3 (Shah et al., 2024) is a memory-efficient attention kernel designed specifically for FP8 arithmetic. It employs a fused softmax and load-store operation organized into a two-stage pipeline to maximize performance. In our work, we extend FlashAttention-3 by introducing a three-stage pipeline for the attention kernel, enhancing memory utilization and compute efficiency. Additionally, we generalize kernel support to mixed-precision $\text{INT4} \times \text{FP8}$ GEMM operations, further optimizing performance for both linear layers and the prefill phase of the attention mechanism.

3 Method

In this section, we present the detailed design of FireQ, a co-designed framework combining INT4-FP8 quantization and kernel-level optimizations to accelerate LLM inference. We first introduce our

optimized INT4-FP8 GEMM kernel and quantization strategy. Then, we describe the three-stage pipelined kernel for the prefill phase and RoPE-aware quantization strategy for attention layers.

3.1 Linear Layer

FireQ adopts a co-designed mixed-precision approach, quantizing linear layer weights to INT4 and activations to FP8. Specifically, weights are quantized using per-token group-wise symmetric quantization with groups of 128 elements sharing a single FP8 (E4M3) scaling factor. Activations are quantized in FP8 (E4M3) format using a BF16 scaling factor. Section 3.1.1 details the corresponding GEMM kernel implementation.

3.1.1 Kernel optimization: $\text{INT4} \times \text{FP8}$.

Figure 2 illustrates the $\text{INT4} \times \text{FP8}$ GEMM kernel structure, which is applied across all linear layers (e.g., up, down, gate projections, QKV generation, and output projection) as well as for attention score ($\mathbf{S} = \mathbf{QK}^T$) and output ($\mathbf{O} = \mathbf{PV}$) computations (Figure 1).

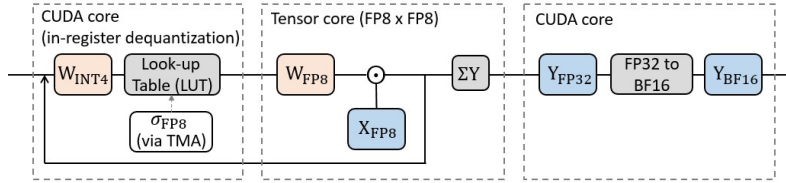


Figure 2: $\text{INT4} \times \text{FP8}$ GEMM kernel for linear layer computations.

We implement this optimized kernel using CUTLASS library through three steps (Figure 2):

- Step 1: A lookup table (LUT) of 16 FP8 entries is constructed based on the group-wise FP8 scaling factor σ , mapping INT4 values $\{-8\sigma, -7\sigma, \dots, 6\sigma, 7\sigma\}$. INT4 weights and corresponding FP8 scaling factors are asynchronously loaded into shared memory using the Tensor Memory Accelerator (TMA), and INT4 weights are converted to FP8 in registers by CUDA cores.
- Step 2: The tensor cores perform GEMM by multiplying FP8 input activations with FP8-converted weights, accumulating intermediate partial sums in FP32 to minimize precision loss.
- Step 3: CUDA cores reduce GEMM outputs from FP32 to BF16 for efficient epilogue computation. This includes activation addition, nonlinear activation functions (e.g., ReLU, SiLU), and element-wise multiplications. Leveraging Hopper GPUs’ optimized FP16/BF16 computation intrinsics and assemblies, this step maximizes throughput.

The selection of (σ_{FP8}) is a key contribution of FireQ for accelerating inference. A detailed experimental analysis is provided in Section 4.2, and the rationale is discussed in Appendix A.

3.1.2 Linear layer scaling strategy

Although the FP8 scaling factor σ_{FP8} improves throughput in quantized model serving, it may introduce underflow and overflow issues, as described in Appendix B. To address underflow caused by FP8 scaling factors (σ_{FP8}) and enhance INT4 quantization accuracy, we propose two scaling strategies: **channel-wise absmean scaling (CAS)** and **per-tensor scaling (PTS)**. **Channel-wise absmean scaling** normalizes each channel’s distribution to a shared target absolute mean, reducing inter-channel variance and improving group quantization stability.

Definition 3.1 (Channel-wise Absmean Scaling, CAS). Let $\mathbf{W}$ be a weight matrix, and $\mathbf{\Lambda}$ a diagonal scaling matrix. The scaled weight matrix is given by:

$$\bar{\mathbf{W}} = \mathbf{W}\mathbf{\Lambda}, \quad \text{and} \quad \lambda^i \triangleq \frac{\bar{\omega}}{\text{absmean}(\omega^i)}, \quad (6)$$

where λ^i is the i -th diagonal element of $\mathbf{\Lambda}$, ω^i is the i -th channel of $\mathbf{W}$, and $\bar{\omega}$ the target absmean.

To maintain computational equivalence, we apply the inverse scaling to activations:

$$\mathbf{Y} = (\mathbf{X}\mathbf{\Lambda}^{-1})_{\text{FP8}} \cdot (\mathbf{\Lambda}^T \mathbf{W}^T)_{\text{INT4}} = \mathbf{X}\mathbf{W}^T, \quad (7)$$

where $(\cdot)_{\text{precision}}$ indicates the representation precision of the tensor. As illustrated in Figure 1, similar to how Λ is merged offline with weights prior to quantization, the inverse scaling factor Λ^{-1} can also be integrated offline with the preceding layer’s weights, enabling offline merging and quantization. In Figure 1, for Λ_1 , λ^i is exceptionally set as a constant for quantization stability.

To further mitigate underflow risks, we introduce **per-tensor scaling**, globally adjusting the scale of weights. We choose the smallest scaling factor $\delta = 2^n$ to balance underflow prevention and overflow risk as follows:

Definition 3.2 (Per-tensor Scaling for Preventing Underflow, PTS). *Let $\mathbf{W}$ be a weight matrix. We define n as the smallest non-negative integer meeting one of the following conditions:*

1. For all $i \in \mathbb{N}$,

$$\mathbb{S}(\mathbf{W} \cdot 2^n) = \mathbb{S}(\mathbf{W} \cdot 2^{n+i}), \quad \text{and} \quad \mathbb{S}(\mathbf{W}) \triangleq \sum_{\omega \in \mathbf{W}} \max(0, 7 \cdot 2^{-9} - |\omega|) \quad (8)$$

measures cumulative underflow relative to FP8’s minimum representable value.

2. There exists an element $\omega \in \mathbf{W}$ satisfying:

$$7 \cdot 2^{5-n} \leq |\omega| < 7 \cdot 2^{6-n}, \quad (9)$$

indicating overflow risk at scaling beyond $\delta = 2^n$.

Per-tensor scaling ensures that as many weight elements ω as possible fall within the representable range of FP8. The weight matrix $\mathbf{W}$ is scaled by δ before quantization, and the inverse scaling δ^{-1} is simply multiplied to the GEMM output. A detailed derivation is provided in Appendix C.1.

3.2 Attention Layer

3.2.1 RoPE-aware quantization and outlier smoothing

To efficiently quantize attention layers, FireQ quantizes query vectors to FP8 and key-value pairs to INT4. However, quantizing key-value matrices poses unique challenges due to the rotary positional embeddings, which significantly alter the statistical distributions of query and key vectors. Figure 3 illustrates various RoPE-aware quantization strategies for the key matrices.

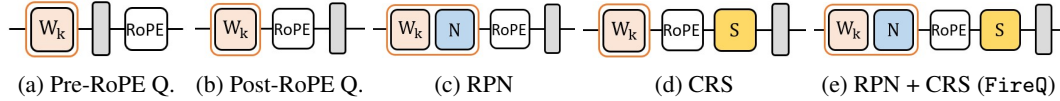


Figure 3: RoPE-aware quantization and outlier smoothing strategies for key matrix.

Post-RoPE quantization. FireQ adopts post-RoPE quantization (Figure 3b) as a baseline, applying per-token (row-wise) quantization where each row has its scaling factor σ . In contrast, pre-RoPE quantization (Figure 3a) requires additional quantization and dequantization steps, because the quantized key cache must be dequantized before applying RoPE, and then re-quantized afterward. Post-RoPE quantization reduces overhead by eliminating these extra operations. Nonetheless, designing effective outlier smoothing under post-RoPE quantization is not straightforward. To address this, we propose a two-stage strategy (Figure 3e) that combines both pre- and post-RoPE smoothing techniques.

Two-stage RoPE-aware outlier smoothing. As RoPE intermixes paired channel distributions while leaving non-paired channels unaffected, we separate key matrix channels into two categories: (i) outlier channels with their paired counterparts, and (ii) regular channel pairs. We apply Channel-wise RoPE Scaling (CRS) post-RoPE to the first group, and RoPE-preserving normalization (RPN) pre-RoPE to the second group.

Stage 1. RoPE-preserving normalization (RPN): For each RoPE channel pair $(\mathbf{k}_i, \mathbf{k}_j)$ within a head, RPN calculates the maximum L_2 norm across tokens and applies a common scaling factor ensuring a bounded norm of post-RoPE (Theorem 3.1). As a linear, offline-calibrated transformation, RPN can be fused into model weights without runtime overhead (Figure 3c and Figure 3e), improving dynamic range utilization and reducing INT4 quantization clipping risks.

Theorem 3.1 (RoPE-preserving Normalization Bound). *Given a key matrix $\mathbf{K} \in \mathbb{R}^{N \times d}$, where N is the sequence length and d is the head dimension, consider a RoPE channel pair*

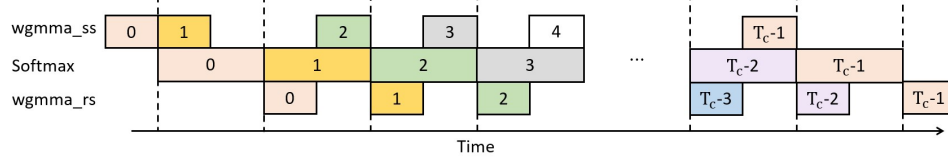


Figure 4: 3-stage `wgmma_ss`-Softmax-`wgmma_rs` pipelining.

$(\mathbf{k}_i, \mathbf{k}_j) \in \mathbb{R}^{N \times 2}$ with $j = i + \frac{d}{2}$. Then, for $\alpha > 0$, the scaled key channels satisfy:

$$\left\| \begin{pmatrix} \frac{k_i^n}{s_i}, \frac{k_j^n}{s_j} \end{pmatrix} \right\|_2 \leq \frac{1}{\alpha}, \quad \forall n \in \{1, \dots, N\}, \quad \text{and} \quad s_j = s_i \triangleq \alpha \cdot \max_{n \in \{1, \dots, N\}} \|(k_i^n, k_j^n)\|_2, \quad (10)$$

where s_i (or s_j) is the shared scaling factor of channel i and j . That is, RPN ensures that scaled channel pairs have their norms bounded within a radius of $1/\alpha$.

Theorem 3.1 highlights a distinct advantage of our approach: the uniform boundedness of channel norms of post-RoPE, which reduces inter-channel variance and effectively smooths outliers. A simple proof of this theorem is provided in Appendix C.2.

Stage 2. Channel-wise RoPE Scaling (CRS): CRS applies distinct scaling factors independently to outlier channels and their pairs post-RoPE, effectively smoothing outliers. For outlier channel $\mathbf{k}_i = \{k_i^n\}_{n \in \{1, \dots, N\}}$, the CRS factor t_i is defined by: $t_i \triangleq \beta \cdot \max_{n \in \{1, \dots, N\}} |k_i^n|$ for $\beta > 0$. Unlike RPN, CRS incurs slight online computational overhead (Figure 3d and Figure 3e), but its limited application scope ensures that the overhead remains negligible.

We note that for computational equivalence, the inverse calibration matrices, RPN (N^{-1}) and CRS (S^{-1}), are merged in the query projection layer, as shown in Figure 1.

3.2.2 Modified FlashAttention-3: prefill phase

To further optimize inference performance, FireQ extends the original two-stage pipeline of FlashAttention-3 (Shah et al., 2024) to a three-stage pipelined approach, as depicted in Figure 4. The overall procedure of prefill phase consists of two main warpgroup components: the Producer and the Consumer, each responsible for distinct stages of data processing.

Producer warpgroup (data preparation). The producer warpgroup asynchronously loads the Q, K, and V matrices from global memory (GMEM) to shared memory (SMEM), using the TMA. The value matrix is transposed during loading to optimize subsequent GEMM operations, ensuring minimal latency and efficient data flow for the consumer warpgroup.

Consumer warpgroup (three-stage computation). The consumer warpgroups (Algorithm 1) execute computations asynchronously in three overlapping stages (Figure 4):

- Stage 1. Query-Key multiplication ($\mathbf{S} = \mathbf{QK}^T$) (lines 3 to 8): GEMM operations are performed directly from SMEM using `wgmma_ss`, with results accumulated in FP16. Computed attention scores serve as inputs to Stage 2 without waiting, enabling pipelined execution.
- Stage 2. Softmax computation ($\mathbf{P}_{ij} = \text{softmax}(\mathbf{S}_{ij})$) (lines 9 to 17): Attention scores from Stage 1 are normalized via softmax in FP16 and subsequently quantized to FP8 for efficient storage in registers. Meanwhile, the subsequent query-key multiplication begins concurrently in Stage 1.
- Stage 3. Value aggregation ($\mathbf{O} = \mathbf{PV}$) (lines 18 to 24): The FP8 softmax matrix ($\mathbf{P}_{ij}$) multiplies the transposed value matrix $\mathbf{V}^T$ using `wgmma_rs`, involving register-to-SMEM operations. Intermediate results accumulate in FP32, then reduce to BF16 for final storage back into GMEM.

Each computational stage is executed asynchronously and synchronized by explicit `wait` instructions to ensure correct data dependencies. To optimize memory usage and throughput further, we reduce the precision of the accumulated $\mathbf{S}_{ij}$ values and the row-wise maximum values m_i from FP32 to FP16, utilizing FP16 exponential instructions. Although this precision reduction might introduce minimal accuracy loss, the subsequent FP8 quantization of softmax results $\mathbf{P}_{ij}$ ensures sufficient accuracy for the final GEMM step. We retain FP32 precision only for critical intermediate accumulations (s_i , l_i , and $\mathbf{O}_i$) to preserve accuracy in the final result. We note that a complete algorithm, including the behavior of the producer warpgroup, is provided in Appendix F.

Algorithm 1 Three-stage attention layer forward at prefill phase in consumer warpgroup

Require: $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, query block size B_r , key and value block size B_c with $T_c = \lceil N/B_c \rceil$

- 1: Initialize pipeline objects to manage barrier synchronization with a 2-stage SMEM buffer
 - 2: Fetch the query index i from global memory with atomic decrement
 - 3: Allocate the predetermined number of registers
 - 4: Initialize $\mathbf{O}_{ik} = (0) \in \mathbb{R}^{B_r \times d}$ for $k \in \{0, 1\}$, $s_i, l_i, m_i = (1), (0), (-\infty) \in \mathbb{R}^{B_r}$
 - 5: Commit $\mathbf{S}_{i0} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_0)$ and wait $\mathbf{S}_{i0}$
 - 6: Commit $\mathbf{S}_{i1} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_1)$ and do not wait $\mathbf{S}_{i1}$
 - 7: Compute $\mathbf{S}_{i0} = \text{mask}(\tau \mathbf{S}_{i0})$, $m_i = \max(\text{rowmax}(\mathbf{S}_{i0}))$, $\mathbf{P}_{i0} = \exp(\mathbf{S}_{i0} - m_i)$, $l_i = \text{rowsum}(\mathbf{P}_{i0})$
 - 8: Wait $\mathbf{S}_{i1}$
 - 9: **for** $2 \leq j < T_c$ **do**
 - 10: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(j-2)}, \mathbf{V}_{j-2}^T)$ and do not wait $\mathbf{O}_{ik}$
 - 11: Compute $\mathbf{S}_{i(j-1)} = \text{mask}(\tau \mathbf{S}_{i(j-1)})$
 - 12: $m_{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_{i(j-1)}))$, $s_i = \exp(m_i^{\text{old}} - m_i)$, $l_i = s_i \odot l_i$
 - 13: Commit $\mathbf{S}_{ij} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_j)$ and do not wait $\mathbf{S}_{ij}$
 - 14: Compute $\mathbf{P}_{i(j-1)} = \exp(\mathbf{S}_{i(j-1)} - m_i)$, $l_i = l_i + \text{rowsum}(\mathbf{P}_{i(j-1)})$
 - 15: Wait $\mathbf{O}_{ik}$ and compute $\mathbf{O}_{ik} = s_i \odot \mathbf{O}_{ik}$
 - 16: Wait $\mathbf{S}_{ij}$
 - 17: **end for**
 - 18: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(T_c-2)}, \mathbf{V}_{T_c-2}^T)$ and do not wait $\mathbf{O}_{ik}$
 - 19: $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_{i(T_c-1)}))$, $s_i = \exp(m_i^{\text{old}} - m_i)$, $l_i = s_i \odot l_i$
 - 20: Compute $\mathbf{P}_{i(T_c-1)} = \exp(\mathbf{S}_{i(T_c-1)} - m_i)$, $l_i = l_i + \text{rowsum}(\mathbf{P}_{i(T_c-1)})$
 - 21: Wait $\mathbf{O}_{ik}$ and compute $\mathbf{O}_{ik} = s_i \odot \mathbf{O}_{ik}$
 - 22: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(T_c-1)}, \mathbf{V}_{T_c-1}^T)$ and wait $\mathbf{O}_{ik}$
 - 23: Compute $\mathbf{O}_{ik} = 1/l_i \odot \mathbf{O}_{ik}$
 - 24: TMA store $\mathbf{O}_{ik}$ from SMEM to GMEM
-

4 Evaluation

4.1 Evaluation Setup

We evaluate FireQ on a range of LLMs, including Llama3.1 (8B), Llama3 (8B) (AI@Meta, 2024), Llama2 (7B, 13B) (Touvron et al., 2023). All experiments are conducted on NVIDIA H100 SXM5 GPUs, configured with CUDA 12.6 and PyTorch 2.6.0 on a 2-socket Intel(R) Xeon(R) Platinum 8468 CPUs with 2TB DDR5 memory system by PCIe Gen5 X16 links. All of the experiments are conducted on a single GPU, without using PCIe or NVLink for tensor inter-GPU communications. Kernel optimization for linear layers and prefill phase employs $\text{INT4} \times \text{FP8}$ GEMM operations implemented using the CUTLASS library (NVIDIA, 2017). To assess the accuracy of FireQ, we employ the WikiText-2 (Merity et al., 2016) dataset to measure perplexity (PPL), while for downstream tasks, we adopt six different zero-shot evaluation including PIQA (Bisk et al., 2020), ARC-Easy (Clark et al., 2018), ARC-Challenge (Clark et al., 2018), BoolQ (Clark et al., 2019), HellaSwag (Zellers et al., 2019), and WinoGrande (Sakaguchi et al., 2021). Throughput is evaluated on the FFN layer and the prefill phase, by measuring the number of generated tokens per second, averaged over ten repeated runs. For accuracy and throughput comparisons, we include state-of-the-art quantization methods: INT4-FP8 AWQ (Lin et al., 2024a) implemented with TensorRT-LLM v0.18.2, QServe (Lin et al., 2024b), Atom (Zhao et al., 2024), and QuaRot (Ashkboos et al., 2024).

4.2 Impact of INT4-FP8 Kernel

We investigate the performance improvements provided by FireQ’s INT4-FP8 GEMM kernel by evaluating throughput in feed-forward network layers. As a baseline, we use non-quantized FP16 models evaluated with the vLLM framework (Kwon et al., 2023). Typically, quantized models exhibit reduced throughput compared to non-quantized counterparts due to quantization and dequantization overheads on CUDA cores, especially in GEMM operations. However, as illustrated in Figure 5, FireQ in most cases surpasses the throughput of the non-quantized FP16 baseline. This occurs because FireQ not only minimizes performance penalties associated with dequantization but also benefits from reduced memory accesses enabled by lower-bit quantization. Furthermore, when

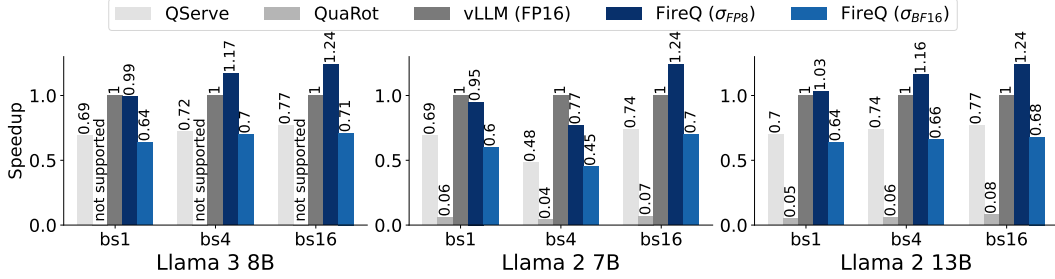


Figure 5: Impact of INT4-FP8 GEMM kernel on feed-forward network layers. The x-axis represents the batch size, with throughput results in the y-axis normalized to the FP16 baseline performance.

compared with the state-of-the-art framework QServe (Lin et al., 2024b), FireQ achieves a 1.68× throughput improvement at the Llama2-8B batch size 16 case. This performance enhancement underscores the effectiveness of our INT4-FP8 GEMM kernel, which employs FP8 scaling factors (σ_{FP8}) for INT4 quantization, enabling efficient in-register dequantization with minimal overhead, thus outperforming QServe’s W4A8-INT operations. Additionally, we include a comparison using BF16 scaling factors (σ_{BF16}) to clearly illustrate the throughput advantage derived specifically from our FP8 scaling approach. A detailed discussion of σ_{BF16} is provided in Appendix B.1.

4.3 Impact of Prefill Phase Optimization

We evaluate the performance of FireQ’s prefill phase kernel, emphasizing improvements in inference throughput. Our benchmarks include the Llama3-8B model tests at a sequence length of 2048 tokens. We compare FireQ against a non-quantized FP16 baseline (TensorRT-LLM FP16) and QServe (Lin et al., 2024b). Given the memory-intensive nature of prefill operations, quantized models generally offer better throughput by reducing memory latency than non-quantized models. Our results demonstrate that FireQ outperforms the comparison frameworks. Specifically, at a sequence length of 1024, FireQ achieves throughput improvements of up to 1.29x over the FP16 baseline and 1.26x over QServe, as described in Table 1. Moreover, at longer sequence lengths (i.e., 2048, 4096), FireQ significantly accelerates first-token generation, achieving 2.53x and 2.38x speedups, respectively, compared to the original FP16 model. Also, the scaled-dot product kernel performance is provided in Appendix D. These improvements primarily arise from our effective utilization of tensor cores for attention computations, contrasting with other frameworks relying on CUDA cores.

Table 1: Prefill phase throughput, Llama3-8B, batch size = 16

	Seq. length	TRT-LLM (FP16)	QServe	FireQ
Tokens/sec (speedup)	1024	28.8k (baseline)	29.4k (1.02x)	37.2k (1.29x)
	2048	13.6k (baseline)	29.4k (2.16x)	34.5k (2.53x)
	4096	13.8k (baseline)	28.5k (2.06x)	32.9k (2.38x)

4.4 Accuracy Evaluation

We evaluate the impact of FireQ on model accuracy to demonstrate that the high-throughput quantization design of FireQ does not compromise inference quality. We benchmark the accuracy of FireQ against both the FP16 non-quantized baseline and several state-of-the-art quantization frameworks, including QServe, Atom, and TensorRT-LLM’s AWQ-based INT4-FP8 models. Our evaluation across standard zero-shot tasks, including PIQA, ARC-e, ARC-c, BoolQ, HellaSwag (HS), and WinoGrande (WG), reveals that FireQ consistently maintains accuracy comparable to other frameworks. Specifically, FireQ achieves minimal accuracy degradation while significantly outperforming other quantized methods in throughput. Notably, FireQ obtains higher average scores on zero-shot benchmarks compared to TensorRT-LLM’s INT4-FP8 framework, and achieves the highest individual scores on PIQA, ARC-e, and WinoGrande benchmarks among all evaluated methods. The perplexity (PPL) of FireQ is approximately 0.3 higher than QServe, but this trade-off is justified by FireQ’s substantial throughput improvements. These results highlight the effectiveness of our carefully

Table 2: Zero-shot accuracy on six common sense tasks for Llama3-8B

Precision	Algorithm	PPL ↓	PIQA ↑	ARC-e ↑	ARC-c ↑	BoolQ ↑	HS ↑	WG ↑	Avg. ↑
FP16	FP16	6.13	79.54	80.09	50.17	81.35	60.18	72.61	70.61
W4A4	Atom	7.57	76.33	77.65	44.28	76.45	55.22	69.37	66.55
W4A8	QServe	6.70	77.80	77.78	48.12	80.49	58.73	71.59	69.08
W4A8-FP	TRT-LLM INT4-FP8	-	77.15	76.64	45.73	73.21	58.44	72.45	67.25
	FireQ (σ_{FP8})	7.06	77.74	78.15	45.64	75.07	58.10	72.77	67.92
	FireQ (σ_{BF16})	7.03	78.62	76.98	45.14	75.44	58.35	71.98	67.75

designed scaling strategies and RoPE-aware quantization, emphasizing FireQ’s ability to deliver superior inference performance without meaningful loss in model accuracy.

4.5 Ablation Study for Quantization Strategies

To thoroughly analyze the impact of our quantization strategies on model accuracy, we conduct an ablation study using the Llama3.1-8B model evaluated on the WikiText2 dataset (Table 3). We systemically address the impact of various outlier smoothing and scaling techniques in FireQ, including RoPE-preserving normalization (RPN), channel-wise RoPE scaling (CRS), channel-wise absmean scaling (CAS), and per-tensor scaling (PTS). Table 3 summarizes the perplexity scores. Individual applications of RPN, CRS, CAS, and PTS each reduce perplexity to varying degrees, highlighting their standalone effectiveness. Combining these strategies yields further improvements, with the comprehensive integration of RPN, CRS, CAS, and PTS achieving the lowest perplexity. The further detailed analysis for these strategies is provided in Appendix E. Additionally, we compare the standard FP8-based FireQ with variants using BF16 scaling factors (σ_{BF16}) and BF16 tensor cores. Using BF16 scaling factors alone marginally improves perplexity over FP8 scaling factors, but introduces a trade-off in inference throughput, as discussed in Figure 5.

Table 3: WikiText2 perplexity of Llama3.1-8B

FireQ variants	No smoothing	RPN	CRS	CAS	PTS	RPN+CRS	RPN+CRS+CAS	RPN+CRS+CAS+PTS
FireQ (σ_{FP8})	8.15	8.00	7.48	7.68	7.98	7.44	7.18	7.11
FireQ (σ_{BF16})	7.87	7.69	7.41	7.50	7.69	7.34	7.09	7.09

5 Related Work

Post-training quantization enhances inference performance for LLMs. SmoothQuant (Xiao et al., 2023) introduces a W8A8 scheme flattening activations during weight quantization, while AWQ (Lin et al., 2024a) proposes a W4A16 strategy quantizing activation-aware weights with FP16 activations. QuaRot (Ashkboos et al., 2024), SpinQuant (Liu et al., 2024), and Atom (Zhao et al., 2024) implement W4A4 quantization using Hadamard transformations to manage outliers in linear and attention layers. Other approaches employ rotations to mitigate outliers in activations (QServe (Lin et al., 2024b), W4A8KV4) or KV caches (KVQuant (Hooper et al., 2024), RotateKV (Su et al., 2025)).

RoPE-aware quantization and KV scaling are extensively explored in frameworks like QServe (Lin et al., 2024b), KVQuant (Hooper et al., 2024), RotateKV (Su et al., 2025), QuaRot (Ashkboos et al., 2024), and SpinQuant (Liu et al., 2024). FireQ combines post-RoPE quantization and pre-RoPE calibration similar to QServe, QuaRot, and SpinQuant for enhanced throughput, and post-RoPE calibration from QServe, KVQuant, and RotateKV for accuracy. It specifically applies channel-wise scaling factors that consider RoPE effects during KV cache calibration.

While previous methods primarily use integer quantization due to constraints of the Ampere architecture, FireQ leverages the Hopper architecture’s FP8 tensor cores by implementing FP8 activations with optimized INT4 $\times$ FP8 kernels. FireQ applies FP8 scaling factors to INT4 quantization, accelerating GEMM operations on FP8 tensor cores. Additionally, it optimizes INT4 $\times$ FP8 kernels using the CUTLASS library and extends FlashAttention-3 (Shah et al., 2024) by upgrading its two-stage pipeline to a three-stage pipeline, significantly enhancing prefill inference throughput.

6 Conclusions

In this paper, we proposed FireQ, an efficient INT4-FP8 quantization and kernel co-design framework tailored for LLM inference on Hopper GPUs. By introducing FP8-aware kernel optimizations and RoPE-aware scaling strategies, FireQ substantially improves inference throughput and alleviates memory bandwidth bottlenecks. In future work, we aim to further enhance inference efficiency by exploring KV cache compression and paging techniques that leverage cache sparsity, yielding additional performance improvements for large-scale LLM development.

References

- AI@Meta. 2024. Llama 3 model card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems* 37 (2024), 100213–100240.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 7432–7439.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044* (2019).
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39. <https://doi.org/10.1109/MM.2024.3373763>
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Hyungdeok Lee, Guhyun Kim, Dayeon Yun, Ilkon Kim, Yongkee Kwon, and Euicheol Lim. 2024. Cost-Effective LLM Accelerator Using Processing in Memory Technology. In *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 1–2. <https://doi.org/10.1109/VLSITechnologyandCir46783.2024.10631397>
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024a. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024b. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532* (2024).
- Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2024. Spinqant: Llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406* (2024).

- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
- NVIDIA. 2017. CUDA Templates for Linear Algebra Subroutines. <https://github.com/NVIDIA/cutlass>
- NVIDIA. 2023. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. <https://github.com/NVIDIA/TensorRT-LLM>
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
- Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv:2104.09864 [cs.CL]*
- Zunhai Su, Zhe Chen, Wang Shen, Hanyu Wei, Linge Li, Huangqi Yu, and Kehong Yuan. 2025. RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations. *arXiv preprint arXiv:2501.16383* (2025).
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems* 6 (2024), 196–209.

A Rationale for Choosing Scaling Factor σ_{FP8}

In Section 3.1.1, we explained how FireQ accelerates linear layer matrix multiplication using a mixed-precision quantization strategy. Selecting the FP8-precision scaling factor σ_{FP8} for INT4 weight quantization is critical for maximizing the inference speed of quantized models. This choice optimally leverages the FP8 tensor cores available in the Hopper architecture, significantly enhancing throughput compared to state-of-the-art methods such as QServe (Lin et al., 2024b). Here, we provide a detailed comparison between FireQ and QServe in linear layer computation:

Table 4: Linear layer matrix multiplication with mixed-quantization strategy

Quantization Strategy				Implementation Details		
	Weight	Scaling factor (σ)	Activation	Dequantization	GEMM + accum.	Re-quantization
FireQ	INT4	FP8	FP8	Lookup-table	FP8 tensor core	CUDA core
QServe	INT4	BF16/INT8 (two-level quant.)	INT8	Weight unpacking + zero point subtraction + weight reordering	INT8 tensor core	CUDA core

As detailed in Table 4, the dequantization process in FireQ is much simpler than QServe. Specifically, FireQ utilizes an in-register lookup table for dequantization, eliminating the need for additional logical operations. In contrast, QServe requires multiple logical operations, including weight unpacking (unsigned INT4 to unsigned INT8 conversion), zero point subtraction (unsigned INT8 to signed INT8 conversion), and weight reordering to ensure register-level parallelism. This operational simplicity significantly contributes to the throughput improvements observed in feed-forward network layers, as shown in Figure 5. The streamlined implementation and associated performance benefits underpin our choice of the σ_{FP8} scaling factor.

A.1 Detailed operational cost comparison

We compare the quantization overhead of FireQ with QServe, a similar W4A8-based quantization method utilizing INT4 weights and INT8 activations. QServe initially performs per-channel quantization on the original BF16 weights $\mathbf{W}_{\text{BF16}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, with input dimension d_{in} and output dimension d_{out} , converting them into INT8 weights $\mathbf{W}_{\text{INT8}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ using BF16 scale factors $\mathbf{s}_{\text{w}} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{W}_{\text{BF16}} \simeq \mathbf{s}_{\text{w}} \mathbf{W}_{\text{INT8}}. \quad (11)$$

It then applies group-wise quantization with group size g to further convert INT8 weights into INT4 weights $\mathbf{W}_{\text{INT4}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, accompanied by corresponding scale factors and zero points $\mathbf{s}_{\text{INT8}}, \mathbf{z}_{\text{INT4}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}/g}$:

$$\mathbf{W}_{\text{INT8}} \simeq (\mathbf{W}_{\text{INT4}} - \mathbf{z}_{\text{INT4}}) \cdot \mathbf{s}_{\text{INT8}}. \quad (12)$$

Moreover, QServe employs per-token quantization for activations, transforming BF16 activations $\mathbf{x}_{\text{BF16}} \in \mathbb{R}^{b \times d_{\text{in}}}$, with batch size b , into INT8 activations $\mathbf{x}_{\text{INT8}} \in \mathbb{R}^{b \times d_{\text{in}}}$ using BF16 scale factor $\mathbf{s}_{\text{x}} \in \mathbb{R}^b$:

$$\mathbf{x}_{\text{BF16}} \simeq \mathbf{s}_{\text{x}} \mathbf{x}_{\text{INT8}}. \quad (13)$$

Consequently, QServe introduces significant overhead for dequantizing linear layer computations $\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{in}} \mathbf{W}^T$, requiring $d_{\text{out}} \times d_{\text{in}}$ INT4 subtractions and INT8 multiplications for dequantizing weights into INT8, as well as $2 \times b \times d_{\text{out}}$ BF16 multiplications for fused dequantization on the output activation.

In contrast, FireQ uses only a group-wise weight scale factor and per-token activation scale factor for quantization, leading to only $d_{\text{out}} \times d_{\text{in}}$ of FP8 register lookups, and $b \times d_{\text{out}}$ times of BF16 multiplications on the output activation. Therefore, compared to QServe, FireQ can reduce the computational overhead for dequantization operations in each linear layer to $(b + d_{\text{in}}) \times d_{\text{out}}$.

B Challenge: Underflow and overflow by σ_{FP8}

Although the FP8 scaling factor σ_{FP8} substantially enhances throughput in quantized model serving, it can also lead to underflow and overflow issues due to FP8’s limited dynamic range compared to BF16. Typically, underflow is more prevalent than overflow since the average magnitude of weight values is usually small and massive outliers rarely occur. Thus, we first address the underflow scenario. Figure 6 provides an example of underflow in a quantization group $W^g \subset W$.

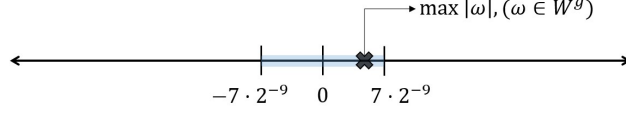


Figure 6: Example of underflow for a quantization group W^g using σ_{FP8} .

Lemma B.1 (Underflow). *If $\max |\omega| < 7 \cdot 2^{-9}$, then all elements $\omega \in W^g$ in the quantization group are INT4-quantized to 0_{INT4} and subsequently FP8-dequantized to 0_{FP8} during the GEMM operation.*

Proof. If $\max |\omega| < 7 \cdot 2^{-9}$, the scaling factor σ_{FP8} is determined as follows:

$$\sigma_{\text{FP8}} = \frac{\max |\omega|}{2^{(b-1)} - 1} = \frac{\max |\omega|}{7} < 2^{-9},$$

where b represents the bit-width for INT4 quantization ($b = 4$). Consequently, the scaling factor becomes zero:

$$\sigma_{\text{FP8}} = 0,$$

as the smallest subnormal number representable by FP8 precision is 2^{-9} . Then, all elements within the quantization group are INT4-quantized to 0_{INT4} and subsequently FP8-dequantized in 0_{FP8} for the GEMM operation. $\square$

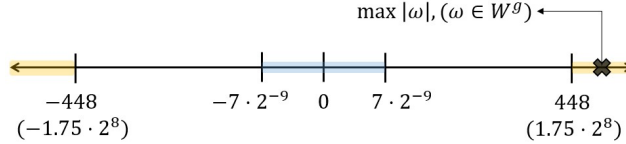


Figure 7: Example of overflow for a quantization group W^g using σ_{FP8} .

Furthermore, Figure 7 illustrates a scenario in which overflow occurs. Although such overflow is rare, it can lead to accuracy degradation when it does occur. FireQ employs per-tensor scaling (PTS) primarily to prevent underflow; however, this approach may inadvertently cause overflow due to the upscaling of weight elements. Thus, identifying conditions under which overflow occurs is crucial. The following section explains how FireQ is designed to simultaneously mitigate underflow and prevent overflow.

B.1 Discussion of σ_{FP8} and σ_{FP16}

We previously discussed that while the FP8 scaling factor improves inference speed, it also triggers underflow and overflow issues that have a negative impact on accuracy. To comparatively illustrate the effects of FP8, we also included an evaluation using a BF16 scaling factor σ_{BF16} . As shown in Figure 5, the feed-forward network layer using σ_{BF16} exhibited a reduced speed, approximately 0.6x that of the σ_{FP8} for Llama-3 8B model. However, Table 2 and 3 show a slight increase in accuracy ($< 0.4\%$) on both PPL and zero-shot accuracy benchmarks with σ_{BF16} . This result implies that adopting an FP8 scaling factor for INT4 quantization on the Hopper architecture achieves substantial speed improvements while incurring only minimal accuracy loss.

C Mathematical Details

C.1 Discussion for Definition 3.2: Per-Tensor Scaling (PTS)

We describe how per-tensor scaling, defined in Definition 3.2, is formulated. The cumulative underflow score $\mathbb{S}(\mathbf{W})$ of weight matrix $\mathbf{W}$ is calculated by summing the distances of all elements $\omega \in \mathbf{W}$ that lie below the underflow threshold $7 \cdot 2^{-9}$, as depicted in Figure 6. Elements above this threshold do not contribute to the score. Formally, the cumulative score $\mathbb{S}(\mathbf{W})$ is defined as:

$$\mathbb{S}(\mathbf{W}) \triangleq \sum_{\omega \in \mathbf{W}} \max(0, 7 \cdot 2^{-9} - |\omega|).$$

A large score $\mathbb{S}(\mathbf{W})$ indicates many elements falling below the threshold. Hence, increasing the scaling factor δ reduces $\mathbb{S}(\mathbf{W})$. The first condition of PTS:

$$\mathbb{S}(\mathbf{W} \cdot 2^n) = \mathbb{S}(\mathbf{W} \cdot 2^{n+i}),$$

implies that FireQ chooses the minimum scaling factor $\delta = 2^n$, beyond which any larger scaling factor $\delta' = 2^{n+i} > \delta$ will not further reduce the cumulative underflow distance for any $i \in \mathbb{N}$.

The second condition of PTS is defined as follows:

$$7 \cdot 2^{5-n} \leq |\omega| < 7 \cdot 2^{6-n},$$

which indicates a risk of overflow if scaling goes beyond $\delta = 2^n$. As depicted in Figure 7, the overflow threshold is $1.75 \cdot 2^8$. Thus, the condition can be rewritten as:

$$1.75 \cdot 2^7 \leq \delta \cdot |\omega| < 1.75 \cdot 2^8 \quad \text{or equivalently,} \quad 7 \cdot 2^{5-n} \leq |\omega| < 7 \cdot 2^{6-n}.$$

This means the sufficiently large scaling factor 2^n is selected such that no element $\omega \in \mathbf{W}$ exceeds the overflow threshold.

C.2 Proof of Theorem 3.1: RoPE-preserving Normalization (RPN) Bound

Proof. Since the shared scaling factor of channel i and j is defined by:

$$s_j = s_i \triangleq \alpha \cdot \max_{n \in \{1, \dots, N\}} \|(k_i^n, k_j^n)\|_2,$$

for all $n \in \{1, \dots, N\}$, the following inequality holds:

$$\frac{\|(k_i^n, k_j^n)\|_2}{\alpha \cdot \max_{n \in \{1, \dots, N\}} \|(k_i^n, k_j^n)\|_2} \leq \frac{1}{\alpha}.$$

□

D Performance of FireQ: Scaled-Dot Product Kernel in Attention Layer

Section 4.3 discussed the overall impact of prefill phase optimization on inference throughput. In this section, we specifically isolate and analyze the performance improvements by the scaled-dot product attention kernel, illustrated in Figure 8. Note that the key and value matrices are quantized in FP8 (not in INT4) for the prefill phase. This is a core component of FireQ’s three-stage pipelined optimization approach described in Algorithm 1.

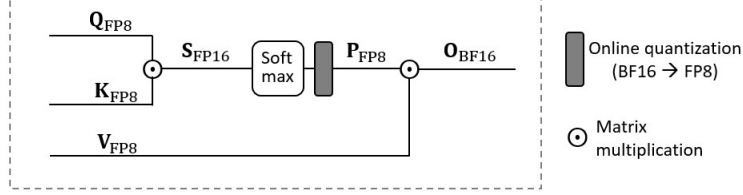


Figure 8: Scale-dot product kernel of FireQ.

Performance evaluation results, depicted in Figure 9, demonstrate that FireQ significantly outperforms both the baseline FlashAttention-2 kernel (without quantization) and the comparative quantization framework, QServe. These results underscore the effectiveness of FireQ’s optimized three-stage pipeline in reducing latency and enhancing inference throughput across varying input sequence lengths for the Llama-3.1 (8B) model.

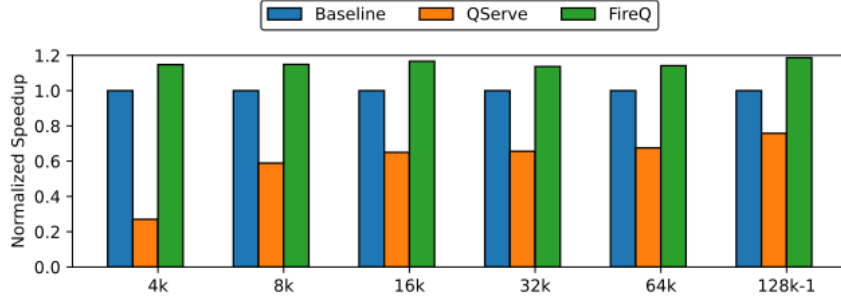


Figure 9: Comparison of scaled-dot product attention kernel performance between FireQ and QServe. The baseline is FlashAttention-2 BF16 kernel. We use the Llama-3.1 8B with varying input sequence length (x-axis).

E Ablation Study for Outlier Smoothing Strategies

E.1 Impact of the Number of Outliers: RPN vs. CRS

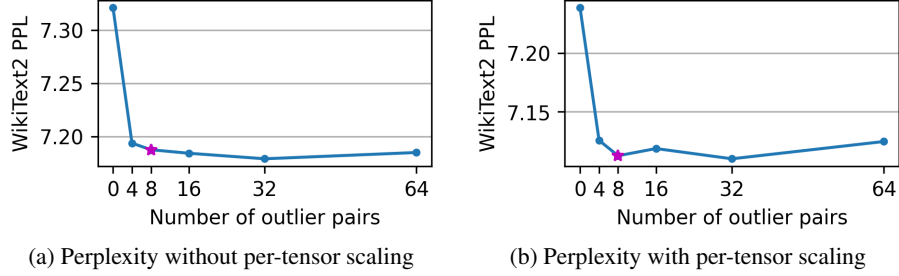


Figure 10: Perplexity on WikiText-2 for the Llama3.1 (8B) model. We select the shared scaling factor for RPN as $\alpha = 8$ and the scaling factor for CRS as $\beta = 8$. Per-tensor scaling is applied only in Figure 10b (right).

As discussed in Section 3.2.1, FireQ categorizes channels in the key matrix into two groups before applying RoPE-aware outlier smoothing: outlier channel pairs (handled by CRS) and normal channels (handled by RPN). Determining the optimal number of outlier channel pairs is crucial since it affects both model accuracy and inference speed, due to the differences in how CRS (computed online) and RPN (merged offline into weights) are implemented. Selecting too many outliers increases the computational runtime overhead from CRS, potentially reducing inference speed. Through calibration (see Figure 11), we identified **eight** as the optimal number of outlier channel pairs. Beyond eight pairs, adding more outliers results in negligible accuracy gain. Thus, FireQ applies CRS to eight outlier channel pairs and RPN to all remaining channels.

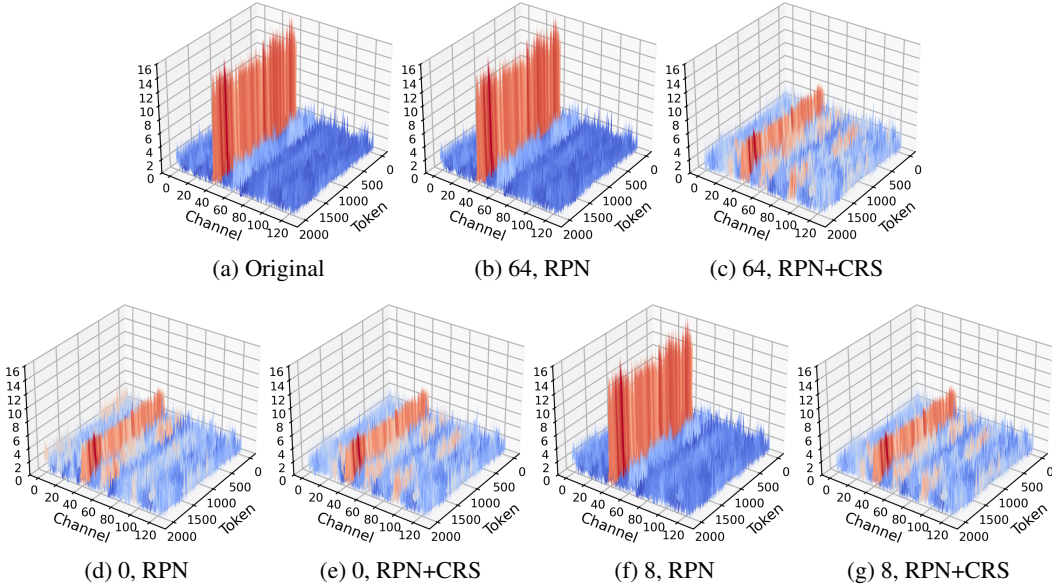


Figure 11: Number of outlier channel pairs, applied smoothing strategies (layer 25, head 6)

E.2 Impact of RPN and CRS on Outlier Smoothing

The key matrix typically contains outliers, as illustrated in Figure 11a. When all channels (64 channels) are designated as outliers, the key matrix remains unchanged after RPN but is effectively smoothed after CRS, as depicted in Figure 11b. Conversely, Figure 11d illustrates the scenario in which no outlier channels are selected, meaning all channels undergo RPN processing. Although the smoothing results from CRS-only (64 outlier channels, see Figure 11c) and RPN-only (0 outlier

channels, see Figure 11e) cases appear visually similar, their performance differs; specifically, the PPL of the CRS-only case is better, as evident in the 64-channel pairs and 0-channel pairs in Figure 11. Finally, when FireQ selects eight outlier channels, Figure 11f shows the key matrix after RPN, and Figure 11g presents the results after applying both RPN and CRS. This approach effectively mitigates the impact of outliers.

E.3 Impact of PTS and CAS on Underflow Mitigation

The per-tensor scaling (PTS) and channel-wise absmean scaling (CAS) strategies are designed to preserve accuracy during linear layer operations, as detailed in Section 3.1.2. CAS normalizes the distribution of each channel toward a shared target absolute mean, while PTS explicitly mitigates underflow by scaling up the weight matrix. Figure 12 illustrates the effectiveness of CAS and PTS across various linear layers, including the up, gate, down, and out projections. The y-axis represents the percentage of quantization groups whose maximum elements fall below the FP8 underflow threshold of $7 \cdot 2^{-9}$, as derived in Appendix B.

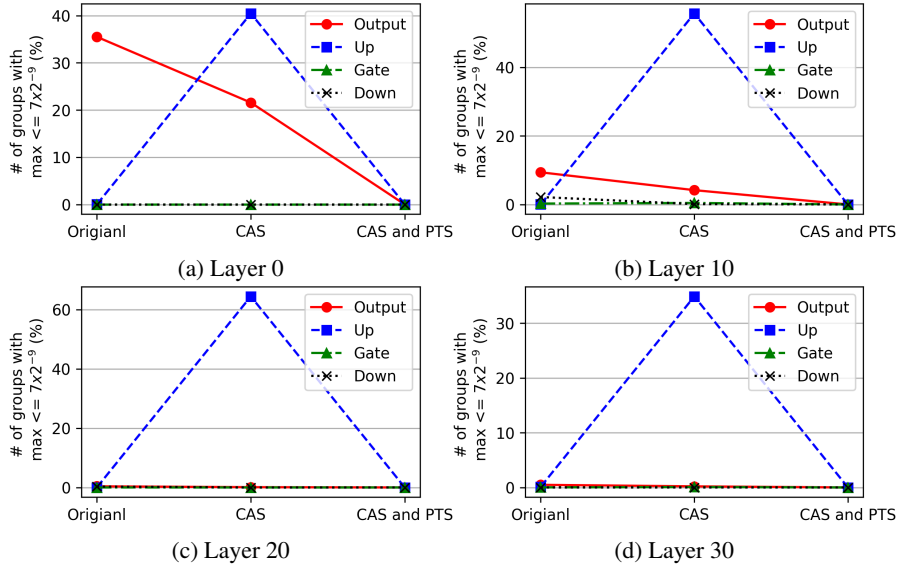


Figure 12: Impact of CAS and PTS on underflow mitigation for different layers.

Figure 12 further demonstrates the reduction of underflow-risk groups in gate, down, and out projections after applying CAS and PTS. Specifically, in the up projection, CAS initially normalizes the values close to the underflow threshold, after which PTS scales these elements upwards. Together, these results confirm that the combination of CAS and PTS substantially mitigates underflow issues and effectively preserves accuracy.

F Modified FlashAttention-3: prefill phase

Due to space constraints, we omitted the data preparation procedure for the `producer warpgroup` in Algorithm 1. Therefore, we provide the complete version of the attention later forward pass during the prefill phase here.

Algorithm 2 Attention layer forward at prefill phase

Require: : Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in GMEM, query block size B_r , key and value block size B_c with $T_c = \lceil N/B_c \rceil$

- 1: Initialize pipeline objects to manage barrier synchronization with a 2-stage SMEM buffer
- 2: Fetch the query index i from global memory with atomic decrement
- 3: **if** `Producer warpgroup` **then**
- 4: Deallocate the predetermined number of registers
- 5: TMA load $\mathbf{Q}_{i0}, \mathbf{K}_0, \mathbf{Q}_{i1}, \mathbf{K}_1$ from GMEM to SMEM in order
- 6: **for** $2 \leq j < T_c$ **do**
- 7: TMA load $\mathbf{V}_{j-2}$ from GMEM to SMEM and transpose to $\mathbf{V}_{j-2}^T$
- 8: TMA load $\mathbf{K}_j$ from GMEM to SMEM
- 9: **end for**
- 10: TMA load $\mathbf{V}_{T_c-2}$ from GMEM to SMEM and transpose to $\mathbf{V}_{T_c-2}^T$
- 11: TMA load $\mathbf{V}_{T_c-1}$ from GMEM to SMEM and transpose to $\mathbf{V}_{T_c-1}^T$
- 12: **else if** `Consumer warpgroup` **then**
- 13: Allocate the predetermined number of registers
- 14: Initialize $\mathbf{O}_{ik} = (0) \in \mathbb{R}^{B_r \times d}$ for $k \in \{0, 1\}$, $s_i, l_i, m_i = (1), (0), (-\infty) \in \mathbb{R}^{B_r}$
- 15: Commit $\mathbf{S}_{i0} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_0)$ and **wait** $\mathbf{S}_{i0}$
- 16: Commit $\mathbf{S}_{i1} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_1)$ and **do not wait** $\mathbf{S}_{i1}$
- 17: Compute $\mathbf{S}_{i0} = \text{mask}(\tau \mathbf{S}_{i0})$, $m_i = \max(\text{rowmax}(\mathbf{S}_{i0}))$, $\mathbf{P}_{i0} = \exp(\mathbf{S}_{i0} - m_i)$, $l_i = \text{rowsum}(\mathbf{P}_{i0})$
- 18: **Wait** $\mathbf{S}_{i1}$
- 19: **for** $2 \leq j < T_c$ **do**
- 20: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(j-2)}, \mathbf{V}_{j-2}^T)$ and **do not wait** $\mathbf{O}_{ik}$
- 21: Compute $\mathbf{S}_{i(j-1)} = \text{mask}(\tau \mathbf{S}_{i(j-1)})$
- 22: $m_{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_{i(j-1)}))$, $s_i = \exp(m_i^{\text{old}} - m_i)$, $l_i = s_i \odot l_i$
- 23: Commit $\mathbf{S}_{ij} = \text{wgmma_ss}(\mathbf{Q}_{ik}, \mathbf{K}_j)$ and **do not wait** $\mathbf{S}_{ij}$
- 24: Compute $\mathbf{P}_{i(j-1)} = \exp(\mathbf{S}_{i(j-1)} - m_i)$, $l_i = l_i + \text{rowsum}(\mathbf{P}_{i(j-1)})$
- 25: **Wait** $\mathbf{O}_{ik}$ and compute $\mathbf{O}_{ik} = s_i \odot \mathbf{O}_{ik}$
- 26: **Wait** $\mathbf{S}_{ij}$
- 27: **end for**
- 28: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(T_c-2)}, \mathbf{V}_{T_c-2}^T)$ and **do not wait** $\mathbf{O}_{ik}$
- 29: $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_{i(T_c-1)}))$, $s_i = \exp(m_i^{\text{old}} - m_i)$, $l_i = s_i \odot l_i$
- 30: Compute $\mathbf{P}_{i(T_c-1)} = \exp(\mathbf{S}_{i(T_c-1)} - m_i)$, $l_i = l_i + \text{rowsum}(\mathbf{P}_{i(T_c-1)})$
- 31: **Wait** $\mathbf{O}_{ik}$ and compute $\mathbf{O}_{ik} = s_i \odot \mathbf{O}_{ik}$
- 32: Commit $\mathbf{O}_{ik} = \mathbf{O}_{ik} + \text{wgmma_rs}(\mathbf{P}_{i(T_c-1)}, \mathbf{V}_{T_c-1}^T)$ and **wait** $\mathbf{O}_{ik}$
- 33: Compute $\mathbf{O}_{ik} = 1/l_i \odot \mathbf{O}_{ik}$
- 34: TMA store $\mathbf{O}_{ik}$ from SMEM to GMEM
- 35: **end if**
